Ex.No.8 - Object Detection using YOLO

Name: M MATHAN KUMAR

RollNo.:230701181

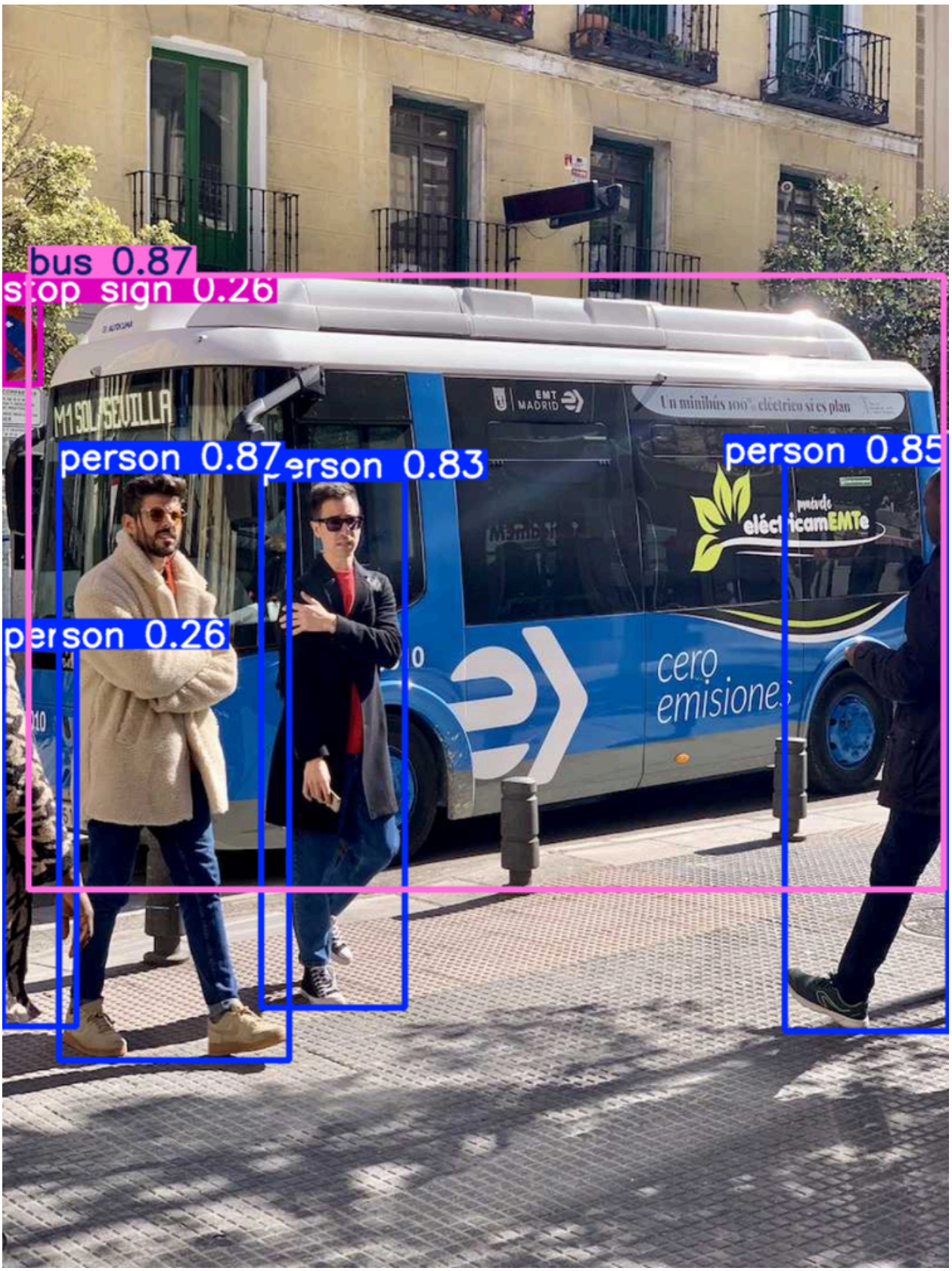
In [12]:
```
import torch
import cv2     as cv
from ultralytics        import YOLO
from IPython.display       import display
from PIL    import Image

print("YOLO installed successfully")
```

```
YOLO installed successfully
```

In [14]:
```
model       = YOLO("yolov8n.pt")
results     = model("https://ultralytics.com/images/bus.jpg")
img  = results[0].plot()         # image with detections
img_rgb   = cv.cvtColor(img, cv.COLOR_BGR2RGB)
display(Image.fromarray(img_rgb))
```

```
Found https://ultralytics.com/images/bus.jpg locally at bus.jpg
image 1/1 C:\Users\Monish\Image Processing and Computer Vision\Lab Experiments\
bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 119.3ms
Speed: 12.8ms preprocess, 119.3ms inference, 0.7ms postprocess per image at sha
pe (1, 3, 640, 480)
```



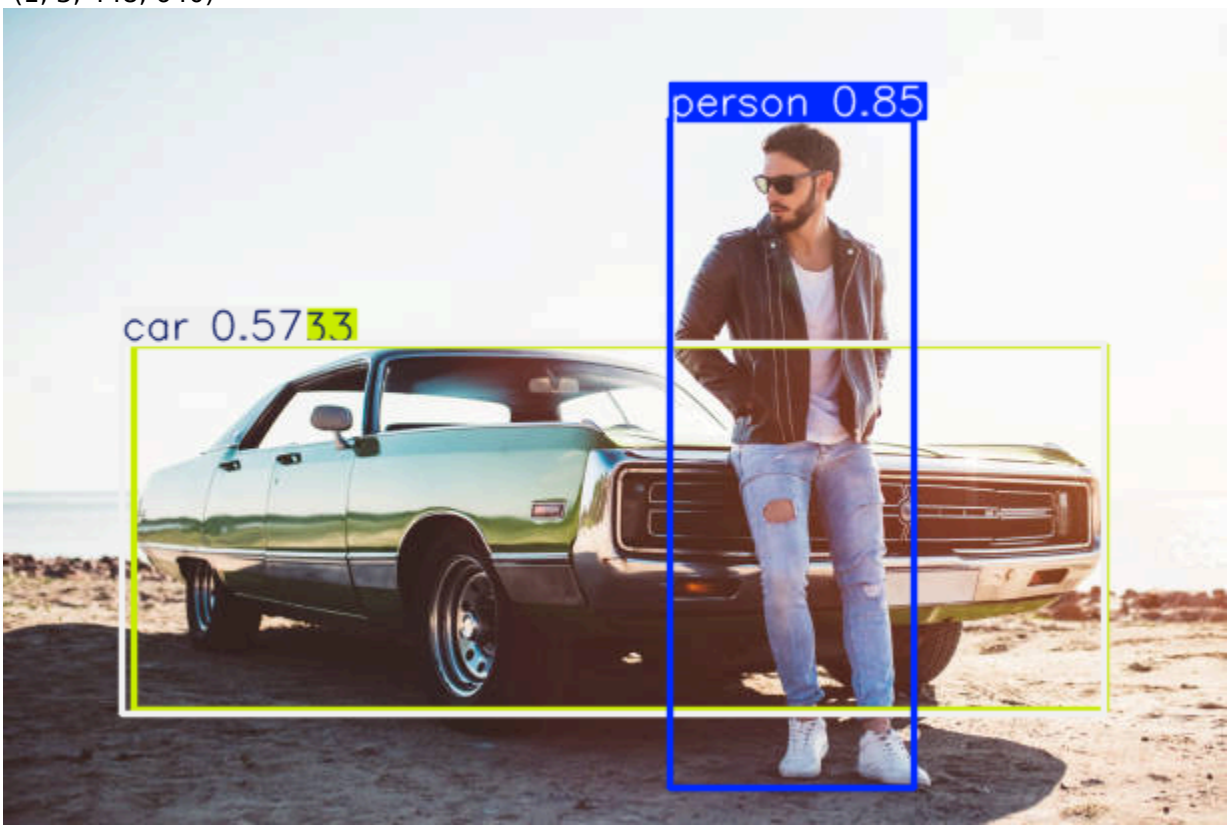
In [17]:

```
# Giving our own input

results    = model(r"D:\VI\IP & CV\Lab Experiments\Inputs\ManCar.jpg")
```

```
img  = results[0].plot()        # image with detections
img_rgb  = cv.cvtColor(img, cv.COLOR_BGR2RGB)
display(Image.fromarray(img_rgb))
```

```
image 1/1 D:\VI\IP & CV\Lab Experiments\Inputs\ManCar.jpg: 448x640 1 person, 1
car, 1 truck, 51.6ms
Speed: 1.3ms preprocess, 51.6ms inference, 0.9ms postprocess per image at shape
(1, 3, 448, 640)
```



Object Tracking

In [19]:
```
import random
import os
from ultralytics        import YOLO
from IPython.display        import Video
```

In [22]:
```
# Load YOLOv8 model
model  = YOLO("yolov8s.pt")

# Use your saved video file
video_path    = r"D:\VI\IP & CV\Lab Experiments\Inputs\Traffic Road.mp4"

cap  = cv.VideoCapture(video_path)
```

In [23]:
```
fps  = int(cap.get(cv.CAP_PROP_FPS))
w = int(cap.get(cv.CAP_PROP_FRAME_WIDTH))
h  = int(cap.get(cv.CAP_PROP_FRAME_HEIGHT))
```

```
# Video writer - saves in Downloads folder
out  = cv.VideoWriter(
    r"D:\VI\IP & CV\Lab Experiments\Outputs\object_detection_output.mp4",
    cv.VideoWriter_fourcc(*'mp4v'),
    fps,
    (w, h)
)
```

In [24]:

```
def get_color(cls_id):
    random.seed(cls_id)
    return tuple(random.randint(0, 255)         for _    in range(3))

# LIMIT: Process only first 100 frames
MAX_FRAME = 100
S                = 0
frame_count
```

In [29]:

```
while
True:
    ret, frame    = cap.read()
    if   not ret:
        break

    # Stop after MAX_FRAMES
    if frame_count     >= MAX_FRAMES:
        print(f"\n✓ Reached limit of          {MAX_FRAMES}frames. Stopping...")
        break

    # Detect objects
    results    = model.track(frame, stream=True, verbose=False)

    for result     in results:
        for box    in result.boxes:

            if box.conf[0]      > 0.4:

                x1, y1, x2, y2      = map(int, box.xyxy[0])
                cls    = int(box.cls[0])
                conf         = float(box.conf[0])
                track_id         = int(box.id[0])  if box.id      is   not  None else 0

                # Draw bounding box
                color    = get_color(cls)
                cv.rectangle(frame, (x1, y1), (x2, y2), color, 2)

                # Draw label
                label    = f"{result.names[cls]} ID:{track_id} {conf:.2f}"

                cv.putText(
                    frame,
                    label,
                    (x1, y1    - 10),
                    cv.FONT_HERSHEY_SIMPLEX,
                    0.6,
```

```
                        color,
                        2
                    )

        out.write(frame)

        frame_count += 1

        if frame_count % 10 == 0:
            print(f"Processed {frame_count}/{MAX_FRAMES} frames...")

cap.release()
out.release()
print("✓ Done! Output saved")

Video(r"D:\VI\IP & CV\Lab Experiments\Outputs\object_detection_output.mp4")
```

```
✓ Done! Output saved
```

Out[29]: Your browser does not support the `video` element.

In [ ]: